# HumAid

Locally relevant Humanitarian Aid knowledge management and alert system

https://humaid.app

# Problem

**The knowledge to save lives already exists**, and is unreachable in the moment it matters. Brazils and Spain floods in recent years. Several in Colombia (Mocoa, La Mojana) every event is documented in hundreds of pages… same lessons get re-learned every La Niña cycle.

**Communication problem:** Risk maps are written in GIS / SAR jargon nobody in the wetland reads. Cloud-based satellite imagery arrives days late and depends on bandwidth that isn't there. Numerical alerts ("Río Putumayo 11.5 m") don't translate to "what do I do?". And the network the cloud-based response runs on is the first thing the flood destroys.

Humanitarian response is information-bound, not only resource-bound. The institutions and the data exist. What's missing is the link that delivers **the right answer to the right person, at the right time** on a device that works without internet.

# Problem

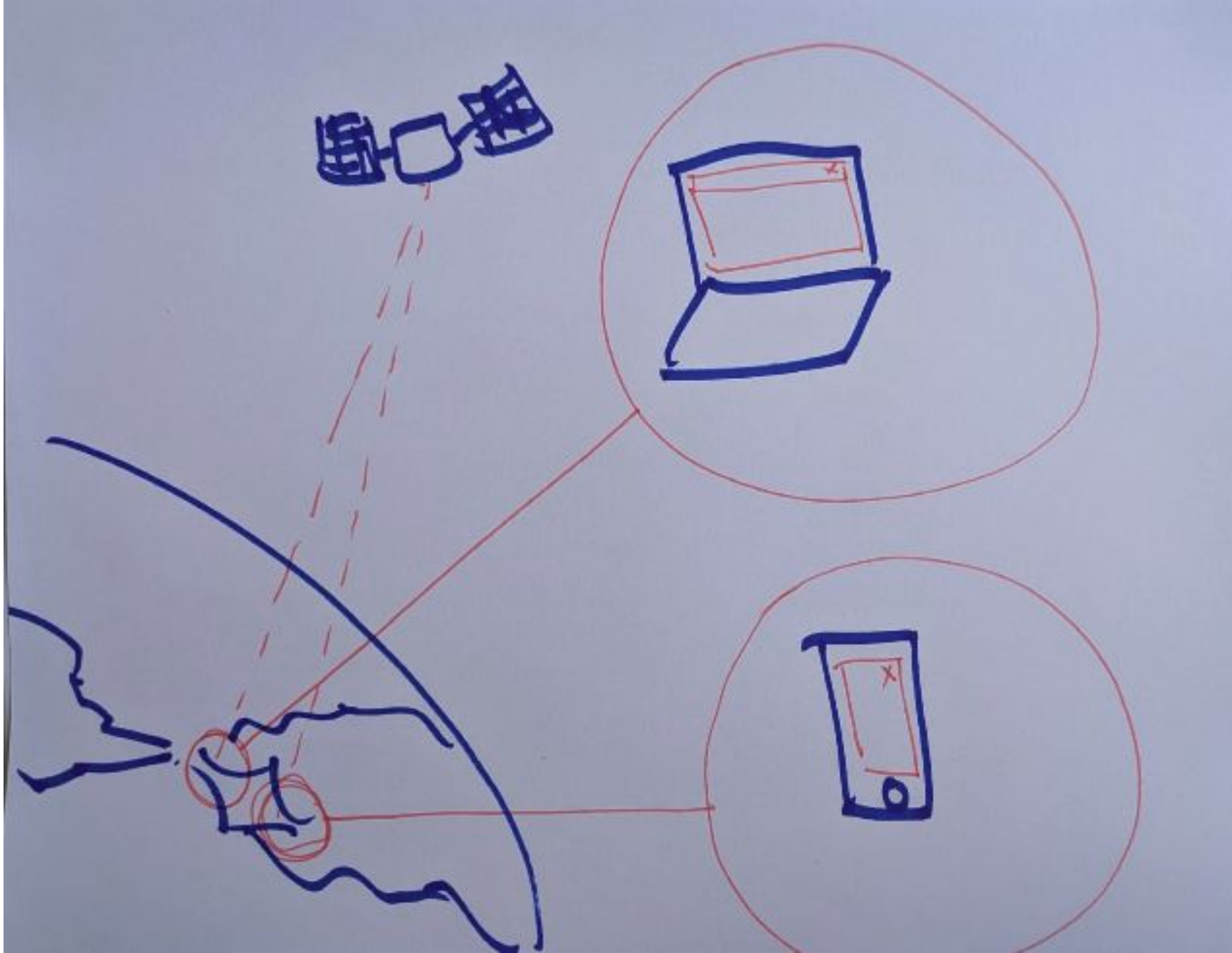
**The knowledge** ... ment it
matters. Brazils a ... ba, La Mojana)
every event is do ... rned every La
Niña cycle.

**Communication** ... dy in the
wetland reads. C ... on bandwidth
that isn't there. N ... what do I do?".
And the network ... destroys.

Humanitarian res ... institutions and
the data exist. W ... **e right**
**person, at the ri**

# Solution

Onboard inference, on-ground delivery. Small VLM on a satellite turns flood imagery into a 200-byte JSON alert… no cloud, no megabytes downlinked.

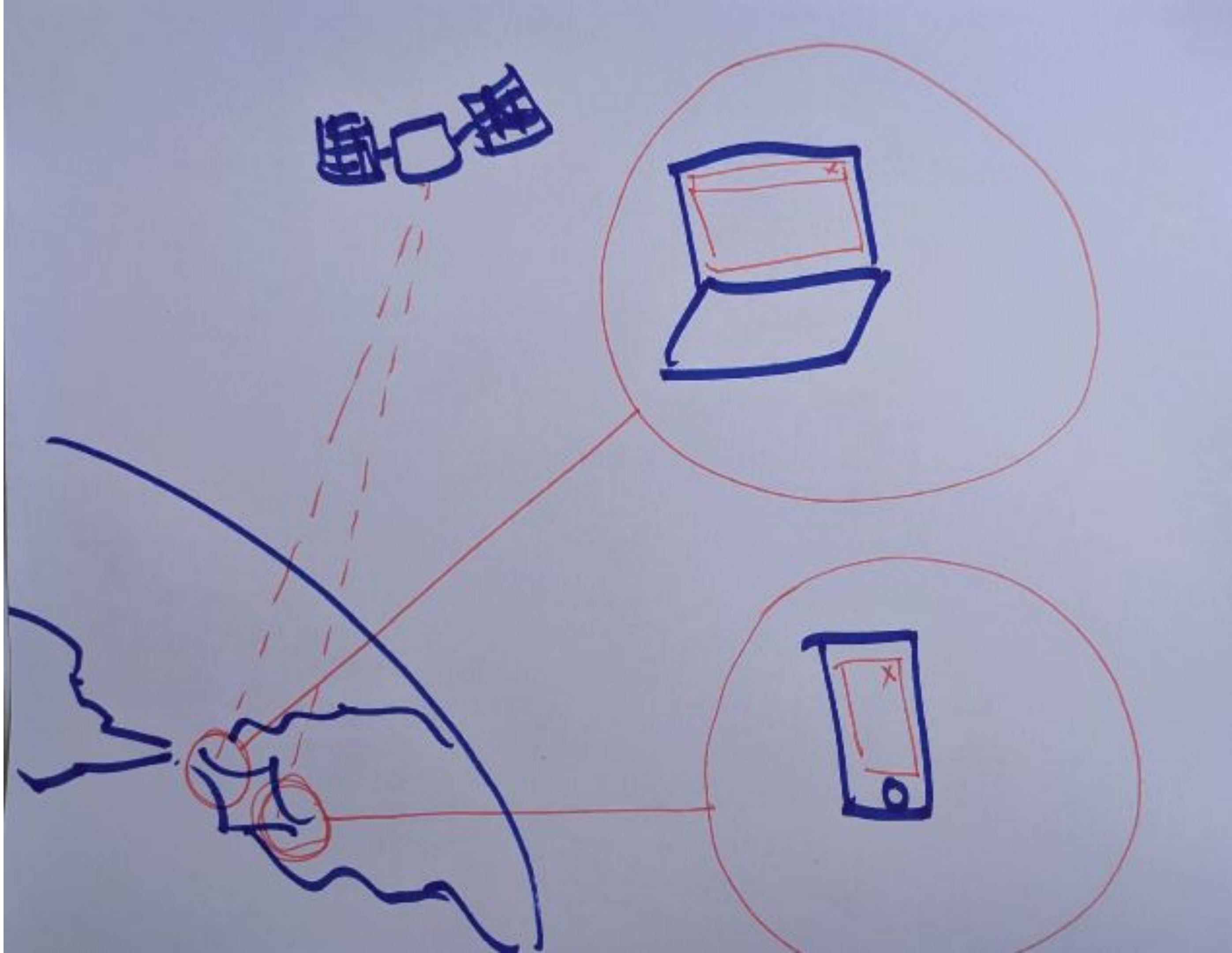


# Solution

Three components, one constraint:

Satellite VLM →
Community sync →
Local app

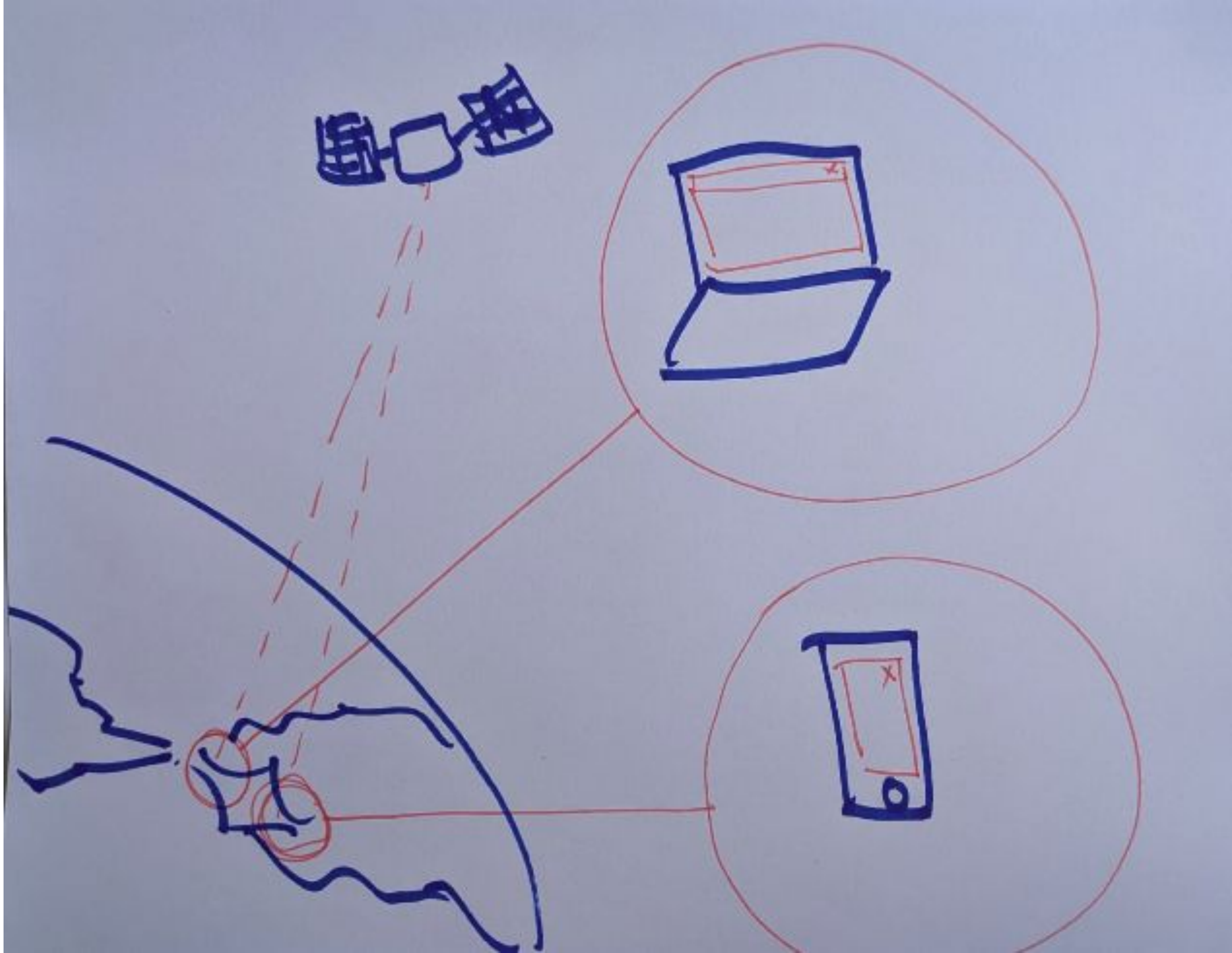
Every link can fail without taking the next on down.



# Solution

Offline-first, role-personalised 471 source-cited knowledge base on device

Retrieved locally with Nomic + DuckDB. Same alert, different answers for campesino vs alcalde vs Red Cross respondent in plain Spanish, with citations, no internet required and context



# Solution

# Architecture

## 1. SPACE (ONBOARD SATELLITE)

### Flood Detection Inference

**LFM2.5-VL-450M** (VLM)
~245 MB Q4_0 GGUF + 50 MB mmproj

4 PNGs in (RGB + SWIR, baseline + current) → {-} JSON

~200 bytes
7-key JSON

**Runtime: llama.cpp / llama-server**

Low-bandwidth ground link

## 2. EARTH (COMMUNITY STATION)

- Receives JSON alerts
- Applies thresholds & dispatch rules
- Hosts kb.duckdb + Q&A content
- Syncs new KB versions (when internet available)
- Serves local network

**Outputs**
- Alerts to LAN
- KB & Q&A
- Documents

**Hardware: Raspberry-Pi class (solar-capable)**

Local Wi-Fi
(No Internet)

## 3. GROUND (LOCAL APP)

### RAG Pipeline (On-Device)

User query (EN / ES)
↓
nomic-embed-text (Ollama)
↓
DuckDB cosine search kb.duckdb (~2.9 MB)
↓
Top-k Q&A (cited) (v1)

(v2) →

### Optional Answer Generation

LFM2 (Text) (Ollama)
↓
Synthesized answer with citations

### User Experience

- Role × Phase × Region
- Alerts feed
- Q&A chat (cited)
- Offline documents (filtered)

**Runtimes: Ollama (nomic-embed-text + LFM2)**

**Data: kb.duckdb (589 rows, 768-dim embeddings inline)**

## MODELS

**LFM2.5-VL-450M (VLM)**
~450 MB (Q4_0 245 MB + mmproj 50 MB)
Satellite-side flood detection.
4 images in → 7-key JSON out.

**LFM2** (Text)
~1.2B class
Answer synthesis from top-k Q&A.
CPU-only inference.

**nomic-embed-text**
137M, 768-dim, multilingual
Embeds questions & queries for cosine retrieval.

## RUNTIMES (WHY TWO?)

**llama.cpp / llama-server (Space)**
- Only runtime that loads LFM2.5-VL mmproj
- OpenAI-compatible local API, tiny footprint

**Ollama (Ground)**
- One-binary install for non-technical users
- Hosts BOTH LFM2 (text) and nomic-embed-text
- Text-only path → mmproj not needed

## DUCKDB INDEX (ON DEVICE)

**kb.duckdb**
~2.9 MB
589 rows (471 humanitarian + 118 project-meta)
Schema: role, phase, region, topic, question_en/es, answer_en/es, references, ref_types, embedding FLOAT[768]

Cosine search:
array_cosine_similarity(embedding, $query_vec)
ORDER BY 1 DESC LIMIT k

**Onboard inference ships answers, not images.** Local retrieval grounds those answers in real PDFs. Two runtimes — llama.cpp in orbit, Ollama on the ground — because the physics is different, but the premise is the same: small, local, grounded. **The cloud is never in the data path.**

# Demo

Website
Desktop app
Android app

Inference
Knowledge base
Question and answer system
Alert System